Домашнее задание №2 по курсу: «Введение в архитектуру вычислительных систем» Вариант 2 - alu_register

Макарская А.
Б01-401
makarskaia.an@phystech.edu
МФТИ ФРКТ

март 2026 г.

1 Отчёт по проделанному заданию

1. **ФИО** Макарская Александра Николаевна
2. **Номер группы** Б01-401
3. **Номер варианта** Вариант 2 - alu_register.
4. **Почта** makarskaia.an@phystech.edu
5. **Ответы в соответствии с заданием:**

5.а Документация для разработанного модуля

Функциональное описание: Модуль `alu_register` при активном сигнале валидности `valid_i` выполняет арифметические и логические операции над двумя входными операндами (`first_i`, `second_i`) в соответствии с кодом операции (`opcode_i`). Результат операции сохраняется во внутренний регистр и выводится на порт `result_o` на следующий тактовый сигнал. Регистр результата не сбрасывается. Сигнал `valid_o` представляет собой задержанную на один такт версию сигнала `valid_i` и говорит о том, что значение на выходе `result_o` является валидным результатом операции, выполненной в предыдущем такте. Модуль поддерживает асинхронный сброс по отрицательному фронту (активный уровень — низкий) `arstn_i`, который обнуляет только регистр валидности (`valid_o`), регистр результата сбросу не подлежит. Модуль поддерживает параметр `WIDTH`, определяющий разрядность операндов и результата.

Структурная схема:

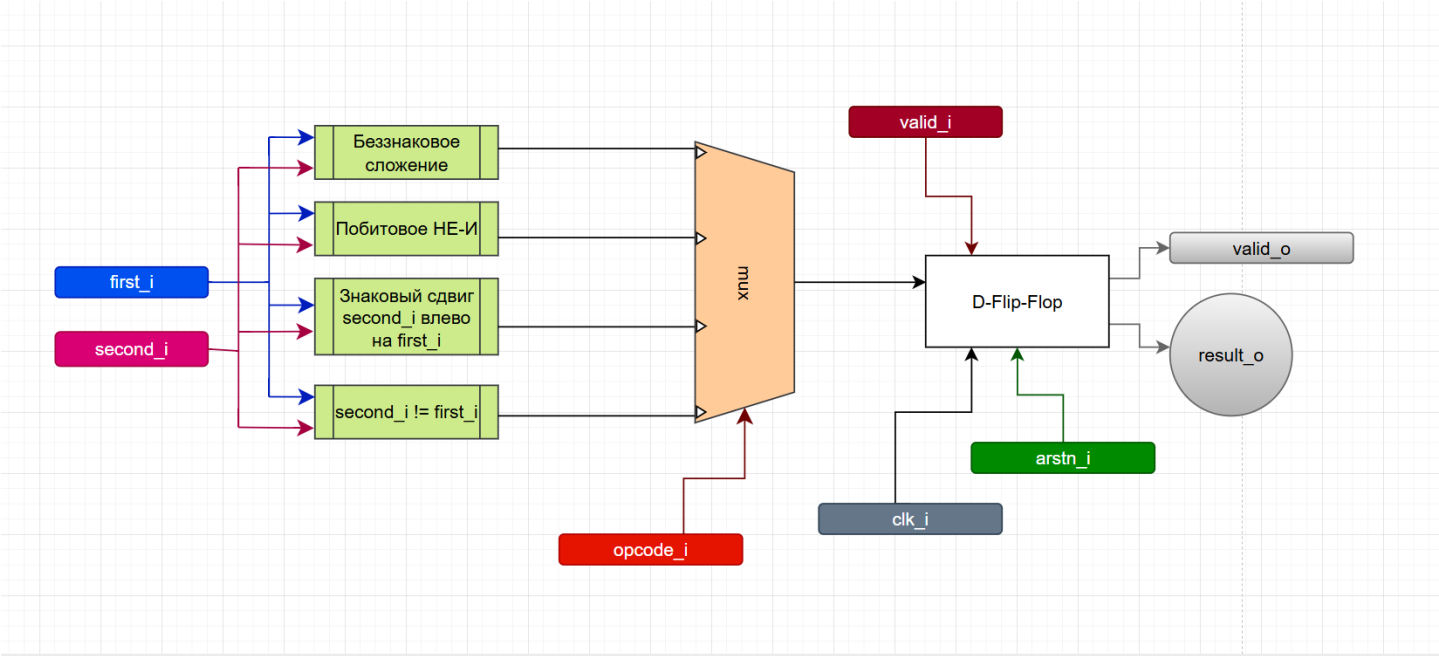

Рис. 1: Структурная схема alu_register в draw.io

Параметры:

Параметр	Описание	Допустимые значения
`WIDTH`	Ширина (разрядность) операндов и результата	Целое число ≥ 1

Таблица 1: Параметры модуля alu_register

Порты:

Название	Ширина	Направление	Описание
`clk_i`	1	input	Тактовый сигнал
`arstn_i`	1	input	Сигнал сброса
`valid_i`	1	input	Сигнал валидности операции
`first_i`	`WIDTH`	input	Шина первого операнда
`second_i`	`WIDTH`	input	Шина второго операнда
`opcode_i`	2	input	Шина, кодирующая операцию
`valid_o`	1	output	Сигнал валидности результата
`result_o`	`WIDTH`	output	Шина результата операции

Таблица 2: Порты модуля alu_register

Тактирование и сброс: Тактирование производится по положительному фронту сигнала `clk_i`. Сброс является асинхронным, активный уровень — 0 `arstn_i` = 0. При сбросе регистр валидности `valid_o` обнуляется, регистр результата сохраняет своё последнее значение.

Тестирование: Сценарий тестирования разработан таким образом, чтобы полностью покрыть все требования технического задания. Были проверены:

1. Асинхронный сброс: проверка обнуления `valid_o` при `arstn_i = 0`.
2. Операция сложения (`opcode = 2'b00`) — значения `0x12 + 0x34`.
3. Побитовое НЕ-И (`opcode = 2'b01`) — проверка инверсии конъюнкции.
4. Арифметический сдвиг влево (`opcode = 2'b10`) — сдвиг на 2 разряда.
5. Сравнение на неравенство (`opcode = 2'b11`) — случаи равенства и неравенства операндов.
6. Поведение при `valid_i = 0` — проверка, что результат и `valid_o` не изменяются.
7. Повторный сброс во время работы модуля.

Все проверки выполнялись с контролем временных соотношений: результат и `valid_o` должны появляться строго на следующем такте после активации `valid_i`.

5.b Имплементация разработанного модуля

```
module alu_register #(
    parameter WIDTH = 8
) (
    input wire          clk_i,
    input wire          arstn_i,
    input wire          valid_i,
    input [WIDTH-1:0]   first_i,
    input [WIDTH-1:0]   second_i,
    input [1:0]         opcode_i,
    output reg          valid_o,
    output [WIDTH-1:0]  result_o
);

    reg [WIDTH-1:0] alu_out;
    reg [WIDTH-1:0] result_reg;

    always @(*) begin
        case (opcode_i)
            2'b00: alu_out = first_i + second_i;
            2'b01: alu_out = ~(first_i & second_i);
            2'b10: alu_out = second_i <<< first_i;
            2'b11: alu_out = {{WIDTH-1{1'b0}}, (second_i != first_i)};
            default: alu_out = {WIDTH{1'b0}};
        endcase
    end

    always @(posedge clk_i)
    begin
        if (valid_i)
            result_reg <= alu_out;
    end

    always @(posedge clk_i or negedge arstn_i)
    begin
        if (!arstn_i)
            valid_o <= 1'b0;
        else
            valid_o <= valid_i;
    end

    assign result_o = result_reg;

endmodule
```

Листинг 1: ALU register module

5.c testbench для разработанного модуля

```
`timescale 1ns/1ps

module alu_register_tb;
    parameter WIDTH = 8;

    reg               clk = 1'b0;
    reg               arstn;
    reg               valid_i;
    reg [WIDTH-1:0]   first;
    reg [WIDTH-1:0]   second;
    reg [1:0]         opcode;
    wire              valid_o;
    wire [WIDTH-1:0]  result;

    alu_register #(.WIDTH(WIDTH)) dut (
        .clk_i(clk),
        .arstn_i(arstn),
        .valid_i(valid_i),
        .first_i(first),
        .second_i(second),
        .opcode_i(opcode),
        .valid_o(valid_o),
        .result_o(result)
    );

    always begin
        #5 clk = ~clk;
    end

    initial begin
        $dumpfile("alu_reg.vcd");
        $dumpvars(0, alu_register_tb);
    end

    initial begin
        clk     = 1'b0;
        arstn   = 1'b0;
        valid_i = 1'b0;
        first   = 0;
        second  = 0;
        opcode  = 0;

        #20;
        arstn = 1'b1;
        #10;

        opcode  = 2'b00;
        first   = 8'h12;
        second  = 8'h34;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;
```

```

        opcode  = 2'b01;
        first   = 8'b11001100;
        second  = 8'b10101010;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;

        opcode  = 2'b10;
        first   = 8'd2;
        second  = 8'b10110010;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;

        opcode  = 2'b11;
        first   = 8'hAA;
        second  = 8'h55;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;

        first   = 8'hAA;
        second  = 8'hAA;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;

        opcode  = 2'b00;
        first   = 8'h01;
        second  = 8'h02;
        valid_i = 1'b1;

        #20;
        valid_i = 1'b0;
        #10;

        first  = 8'hFF;
        second = 8'hFF;

        #20;
        #10;

        arstn = 1'b0;
        #20;
        arstn = 1'b1;
        #20;
```

```

        $finish;
    end

endmodule
```

Листинг 2: ALU register test bench

5.d Временные диаграммы, полученные с помощью разработанного testbench

На временных диаграммах сигналы управления (`clk_i`, `arstn_i`, `valid_i`, `valid_o`, `opcode_i`) я выделила фиолетовым цветом, а шины данных (`first_i`, `second_i`, `result_o`) — зелёным, чтобы улучшить читаемость. Диаграмма демонстрирует задержку результата и сигнала `valid_o` ровно на один такт, сохранение значения `result_o` при `valid_i` $= 0$, а также корректную работу асинхронного сброса. Тесты подтвердили корректность сложения, NAND, сдвига и логического сравнения, что очень радует.

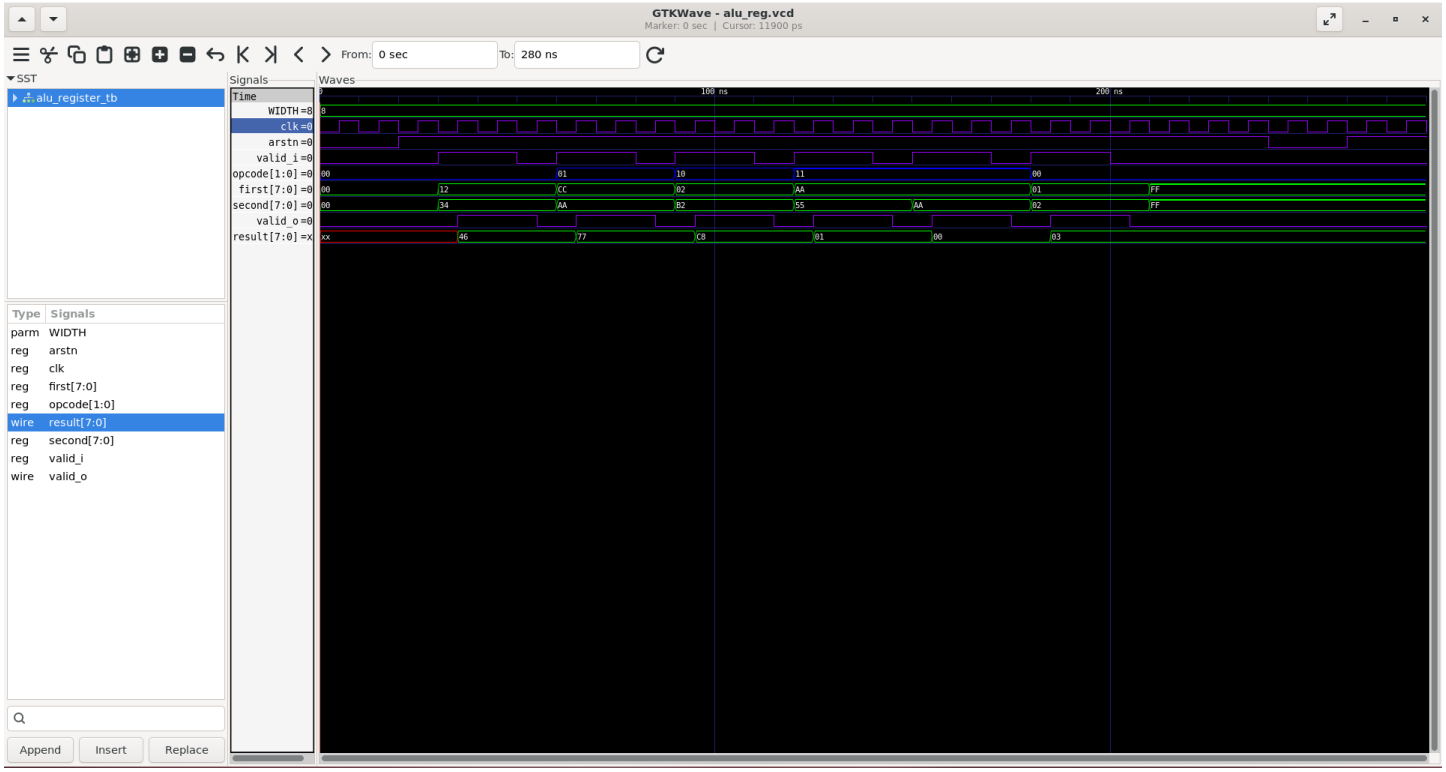

Рис. 2: Временная диаграмма разработанного модуля

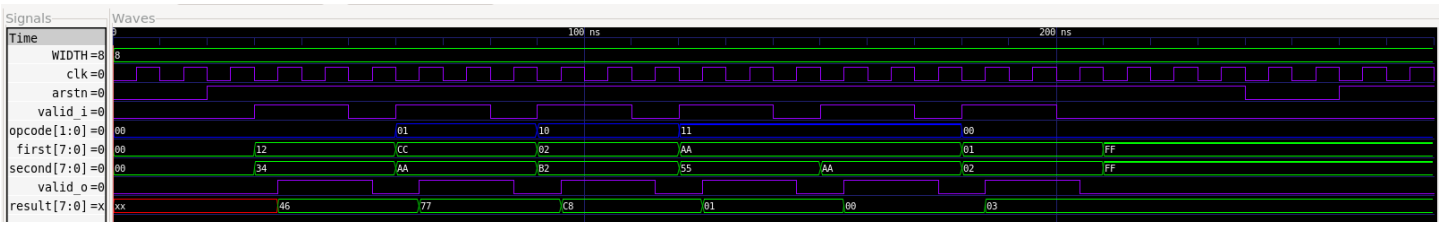

Рис. 3: Увеличенный фрагмент временной диаграммы